SENTIMENT ANALYSIS OF GAME REVIEWS

By: Gayathri Thirumalai Ramu

Table of Contents

List of Figures

List of Tables

1. Introduction

The goal of sentiment analysis is to classify the sentiments conveyed in text as Positive, Negative or neutral. In video games, player reviews provide valuable insight into the quality of the game play, graphics, bugs, user experience and overall satisfaction. By analysing game review analysis companies can gather valuable customer feedback to enhance their products. Using state-of-the-art pretrained transformer models and the Hugging Face Transformers library, the project is designing a robust binary sentiment classifier that can automatically determine if the user has provided a positive or negative experience in their review of a game.

This project is being done in Google Colab according to the guidelines from the CI7525 coursework brief.

2. Problem definition and dataset challenges

2.1 Problem statement

The primary task in this coursework is to sentiment classification if game reviews (steam reviews), mainly using transformer based NLP models. Majorly, the sentiment of user-generated reviews is that they may contain informal language usage, abbreviations, sarcasm, spelling differences, and meanings which may depend on context.
The main goal of the project is to develop an NLP model that accurately classifies user-generated game reviews. The model will learn semantic patterns and relationships between the various types of data in the dataset to provide high quality predictions.

2.2 Dataset challenges

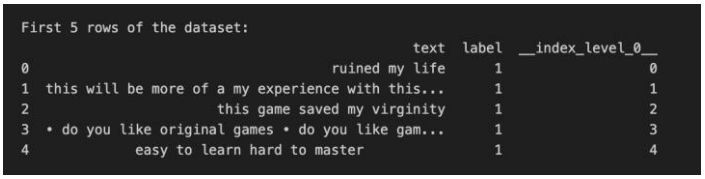
- Steam reviews contain informal language (i.e., slang and abbreviations); have unique gaming terminologies; and contain inconsistent usage of grammar, analysing them with general-purpose models presents unique challenges.
- The dataset is heavily imbalanced towards reviewed games having positive reviews.
- The length of each review varies significantly.

To achieve the goal, two pre-trained models DistilBERT and RoBERTa have been selected so that we can evaluate their applications to this classification task using multi-class evaluation metrics.

3. Dataset selection and Characteristics

3.1 Dataset selection

The dataset for this coursework uses Steam reviews dataset from Hugging face (Adem, 2016) which provides a pre-cleaned corpus of Steam user interactions with sentiment labels. With this dataset, sentiment classification can be done, as it contains many user-generated opinions and emotion-based responses towards the experience of the games.



First 5 rows of the dataset:

	text	label	__index_level_0__
0	ruined my life	1	0
1	this will be more of a my experience with this...	1	1
2	this game saved my virginity	1	2
3	• do you like original games • do you like gam...	1	3
4	easy to learn hard to master	1	4

Figure 1 First 5 rows of the dataset

The dataset was selected because,

- Realistic user-generated texts
- Reviews of varying length and style
- Appropriate for binary classification of sentiments
- Enough data to train transformer models
- Represents a real-world application of NLP

3.2 Dataset structure

The dataset contains almost 4.4M reviews with their sentiment categories. The 2 key columns in the dataset include:

Column	Description
Text	User written game review
Label	Sentiment indicator (originally 1 and -1)

Table 1 Dataset description

3.3 Dataset characteristics

The steam reviews dataset explains several important NLP characteristics.

i. The data has no missing data points in the text or label column; therefore, no rows needed to be imputed or deleted.

ii. The label column is already appropriately mapped, without requiring any need for manual mapping of the recommendations from the original Steam record columns.

iii. **Large volume:** The dataset is made up of thousands of reviews. This allows deep learning models to easily learn useful representations of the language.

iv. **Sentiment distribution:** The distribution of the dataset will determine how many examples there will be of each of the sentiment classes. The numbers of positive and negative labels show a major discrepancy in their counts. 81.5% are positive and only 18.5% are negative. This indicates that the Steam dataset contains a significant amount of 'recommendation' bias against the two classes.

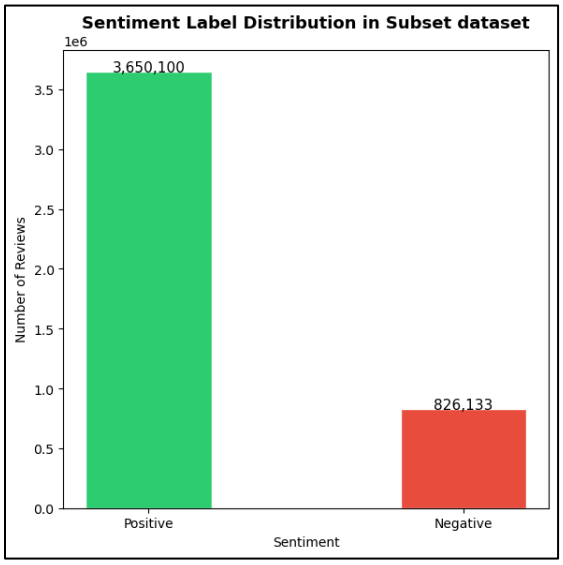


Figure 2 Sentiment Distribution

v. **Variable review length:** Reviews cover a wide range from being short to a maximum length of 3,959 words. negative reviews tend to have more of a long tail in their distribution than positive reviews. These linguistic patterns indicate that because users are unhappy about something, they will generally submit longer, more detailed complaints, compared to shorter ones that express satisfaction with what they received.

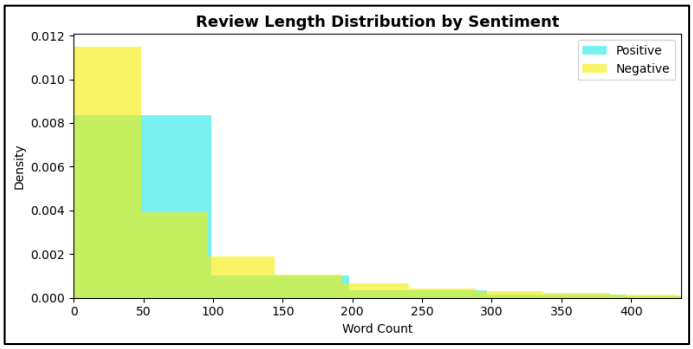


Figure 3 Review length by sentiment

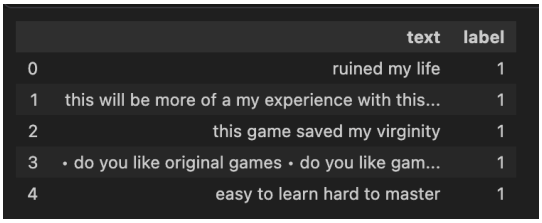
To preserve the constraints of available computing resources, a stratified 20K sample subset of the total dataset was taken. Stratified samples are representative of the distribution of classes (positives and negatives) found within each of the training, validation, and testing data splits. Therefore, being able to maintain the same class distribution across all three splits.

4. Methodologies and Fine-tuning Approach

4.1 Data Preprocessing

Standardised preprocessing is necessary as it ensures the text data is compatible with the Deep learning architectures and keeps the training efficient while performing well in high dimensional feature spaces. A structured pipeline was followed for preprocessing after loading the dataset.

1. **Column selection:** Only 2 columns (text and labels) were kept, all other columns were removed. Below is the head of the dataset after removing unwanted columns.



	text	label
0	ruined my life	1
1	this will be more of a my experience with this...	1
2	this game saved my virginity	1
3	• do you like original games • do you like gam...	1
4	easy to learn hard to master	1

Figure 4 Dataset with only required columns

2. **Label mapping strategy:** In the original dataset, users provide 2 inputs: text and recommendation indicating whether they recommend the game or not. The cleaned version of this dataset given for this project provides a 'label' column where it is encoded as
   - 1 – Positive sentiment
   - -1 – Negative sentiment

   These sentiment labels are not compatible with standard Machine Learning models. So the labels are converted to standard label format for compatibility with Hugging Face's AutoModelForSequenceClassification, which expects non-negative integer class indices. Below is the sentiment column with standard labels.

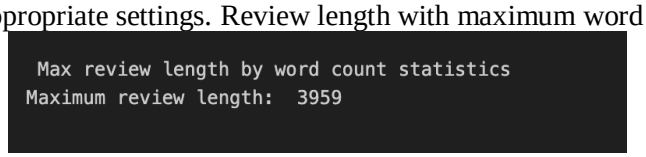
Original label	Mapped label	Sentiment
1	1	Positive
-1	0	Negative

Table 2 Mapping strategy

```
sentiment
1    3650100
0     826133
Name: count, dtype: int64
```

Figure 5 Standard sentiment labels

3. **Review length analysis:** The maximum length of the review text was analysed. Knowing how long reviews are, allowed us to determine appropriate parameters for tokenising and truncating transformer training models to develop the appropriate settings. Review length with maximum word count obtained is 3959.



```
Max review length by word count statistics
Maximum review length:  3959
```

Figure 6 Maximum review length

4.2 Model selection

Two pretrained transformer models were selected for fine-tuning.

i. Model 1: DistilBERT

DistilBERT (distilbert-base-uncased) is a smaller, faster, and more efficient version of BERT (Sanh et al., 2019). With a roughly 40% size reduction and a 60% increase in speed compared to the original BERT model, DistilBERT performs at about 97% of the original BERT model performance level. Therefore, DistilBERT serves as an efficient baseline for this comparison.

ii. Model 2: RoBERTa

RoBERTa (Robustly Optimised BERT Pretraining Approach) (roberta-base) was trained on a much larger dataset than BERT using a dynamic masking technique to improve generalisation (Liu et al., 2019). Additionally, RoBERTa did not use the Next Sentence Prediction objective when training on its data set. RoBERTa has consistently been shown to be superior to BERT when performing sentiment analysis, so it will likely outperform the first Transformer model in this project.

4.3 Tokenisation

Raw text by itself cannot be processed directly using a transformer model. Tokenisation is necessary to convert the text to numerical token IDs for transformer models. The respective tokenisers for each of the models are loaded from Hugging Face's AutoTokenizer. DistilBERT and RoBERTa models were trained with completely different tokenisation process.

- For DistilBERT a WordPiece tokeniser is used.
- For RoBERTa a Byte-Pair Encoding (BPE) tokeniser is used.

Different models use different vocabulary and tokenisation rules, and therefore, need to use their own tokenisers as well to create meaningful input IDs.

i. Define Max_length

To adjust for inconsistencies among review length, we determined MAX_LENGTH empirically from the data by tokenizing a random sample of 10000 reviews and performing a statistical analysis on the lengths of those tokenized reviews.

We took the 95th percentile as our cutoff threshold for maximum length meaning that 95% of the reviews in the dataset contain this length of tokens and thus will be left untruncated. This was a calculated compromise. To have all 100% of the reviews in the data set would require using a MAX_LENGTH value of 512 (the maximum amount of tokens allowed by the model) and therefore, optionally, would unnecessarily pad most reviews because the median number of tokens for a review is only 36. By padding all sequences to 512 tokens when the majority of the reviews only have fewer than 100 tokens, we would have:

- Larger tensors which waste both memory and GPU resources.
- Increased training time due to extra time spent on performing computations over padding tokens that do not have any real relevance or effect on the output from the model.
- Increased risk of crashing memory, especially using larger pre-trained models like RoBERTa.

Thus, we rounded up the p95 value to the next highest multiple of 32 (resulting in MAX_LENGTH = 288). This is an industry standard to provide for better overall performance when training neural networks with tensor dimensions that are aligned to both multiples of 32 and/or 64 because of how GPU architectures align their memory.

This data-driven selection of MAX_LENGTH demonstrates that modelling decisions were grounded in the characteristics of the dataset rather than arbitrary defaults.

ii. Batch tokenisation

Tokenising 4.4 million reviews in one shot crashed the kernel because it tries to allocate massive tensor all at once. To handle the data for 4.4 million reviews, we needed to develop a chunked approach for our tokenization process. In this case, the chunk size of 1,000 samples allows us to process the data over time and avoid consuming excessive memory.

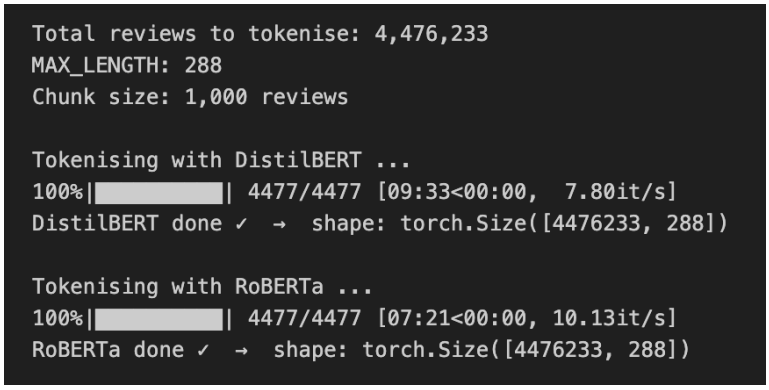


Figure 7 Result after batch tokenisation

4.4 PyTorch dataset construction

The tokenised inputs were then converted into a standardised format (i.e. dictionary of tensors) in accordance with HuggingFace Trainer and PyTorch DataLoader. For the converted tokenised inputs to be served as expected, a custom PyTorch Dataset class is implemented using __len__ and __getitem__.

This provides a means for batches of samples and their associated inputs to be efficiently processed, shuffled and iterated through during both the training and evaluation phases of model development with DataLoader. If there is no standardised structure for the raw tokenised arrays, they cannot be processed by the model's training loop.

4.5 Subset data creation and Data splitting

20,000 samples were selected from the whole dataset as this sample size is faster and more feasible to fine-tune with limited amount of compute resources.

The splitting is made by **index** so the same indices can be applied to both PyTorch datasets. This avoids re-tokenising and keeps DistilBERT and RoBERTa model in sync.

Three splits have been created for training, validation and testing from the subset of 20,000 samples using stratified sampling techniques to maintain the proportions of the class. The same index based splits were performed on both DistilBERT and RoBERTa model, ensuring completely fair comparison between the models.

Split	Proportion	Size
Training	70%	14,000
Validation	15%	3,000
Testing	15%	3,000

Table 3 Data split sets

4.6 Fine tuning setup

Both models were fine-tuned utilizing the Hugging Face 'AutoModelForSequenceClassification' Class which appends a linear classification head (2-node)) on top of each pre-trained transformer backbone. The classification head maps the pooled output from the transformer backbone to produce Binary Logits (Negative / Positive) and the weights for both the classification head and transformer backbone were updated during the training process.

The shared hyperparameters of the two models were kept consistent to maintain a fair and consistent comparison.

Hyperparameter	Value	Reason
Batch size	32	Instead of feeding 14,000 training at once (which would crash memory), we feed 32 reviews at a time
Number of epochs	3	Since transformers are already pretrained, they don't need many rounds to adapt. 3 epoch ()
Learning rate	2e-5	Regulates the size of all weight updates. A very small value (0.00002) as the model has strong pretrained knowledge.
Number of labels	2	We have 2 classes. This value indicates to the neural network the number of output neurons which will be created by the final classification layer.

Table 4 Hyperparameters tuned

- An optimal **learning rate of 2e-5** is considered the norm for fine-tuning of Transformers (Devlin et al., 2019), where the learning rate is set low enough to ensure that it retains its learned representation while also allowing itself to adapt to the current task.
- **AdamW** was used as the optimiser instead of the conventional Adam (Loshchilov and Hutter, 2017) due to the former being a more general approach to weight decay which benefits the transformer model.
- The use of a **linear learning rate scheduler** with warm-up ensured that the learning rate increased initially before starting to decay linearly.

Training loop:

- The training loop consists of batches from the PyTorch DataLoader, which provide a shuffling feature during training at each epoch.
- For each batch, the model performs a forward pass; calculates the cross-entropy loss with respect to the logits: computes backpropagated gradients; and calls step() on both the optimizer and scheduler.
- The average training loss per epoch is tracked in the training loop.

Validation loop:

- The validation loop is run after each training epoch. The model will be switched to the evaluation mode (model.eval())
- The model will be run in inference mode on the validation set with the gradients disabled (torch.no_grad()).
- The validation loss and accuracy are recorded by epoch. Both models demonstrate the highest validation performance at Epoch 1 with the following results:

Model	Best Validation Loss	Best Validation Accuracy
DistilBERT	0.2474	89.77%
RoBERTa	0.2018	92.24%

Table 5 Model Performance

The reason we chose Epoch 1 is because out of 3 epochs, the validation loss is raising every epoch. This means memorising the training data rather than truly learning sentiment patterns. Higher the validation loss means the model is overfitting and unreliable, even if accuracy looks good. So, epoch 1 is chosen result for both the models.

5. Comparative Analysis and Performance Evaluation

5.1 Evaluation metrics

The fine-tuned models were assessed on the withheld test set (3,000 samples) using the metrics calculated from scikit-learn:

- **Accuracy** measures the total proportion of reviews that were correctly classified as either a positive or negative.
- **Macro precision** measures the average overall amount of precision per class but gives both classes equal weight.
- **Macro recall** measures the average overall amount of recall per class, with both classes receiving equal weight. It is critical to measure how accurately the model predicted true positives for both classes.
- **Macro F1** metric calculates the harmonic mean of each class's macro precision and recall score. This represents the most relevant metric due to the imbalance in data between the two classes.
- **Weighted F1** is the aggregated F1 score weighted by how many positive and negative examples exist within each respective category of reviews. Therefore, it represents how well the model will perform in practice, since the classes differ significantly (majority positive).

Confusion matrix:

This outlines how the model classified each review, broken down by the true positives, true negatives, false positives, and false negatives for both classes.

This allows for a visual representation (heatmap) of how accurately the model classified this data into either class.

5.2 Test set results

The results of both models were evaluated using a 3,000-sample test data set. The summary from the notebook shows that RoBERTa outperformed DistilBERT in terms of overall scores.

Metric	DistilBERT	RoBERTa
Accuracy	0.9133	0.9223
Precision (macro)	0.8686	0.8722
Recall (macro)	0.8324	0.8686
F1 (macro)	0.8488	0.8704
F1 (weighted)	0.9111	0.9221

Table 6 Test results

5.3 Observations and Analysis

i. The superior performance of **RoBERTa** can be attributed to its architecture. According to Liu et al. (2019), the large amount of pre-training data used alongside dynamic masking contributes to richer and more generalizable representations of language.

In addition, RoBERTa does not contain the Next Sentence Prediction task which introduces training noise to downstream performance.

ii. The relatively competitive performance of **DistilBERT** must be viewed within context. Although DistilBERT is 40% smaller and 60% faster than BERT (Sanh et al., 2019), it attained an accuracy of just around 2.4% less than RoBERTa. Its effectiveness as a smaller version of BERT should be noted.

iii. **Class imbalance effects:**

There is a clear imbalance in the dataset towards positive reviews, as reflected by macro F1 values being lower than the weighted F1 values for both models. The macro F1 will reflect that there was results from fewer instances of the minority negative class during training for both models, thus creating lower recall on the negative class. Through stratified sampling we were able to keep the same ratios of class representations across splits, but due to the underlying issue of class imbalances we were unable to completely remove this issue.

iv. **Analysis of confusion matrix:**

It is anticipated that both models will have a higher number of false positives compared to false negatives because the dataset contains an imbalance majority class, which forces the model to lean towards making more positive predictions.

v. Convergence behaviour:

The graph comparing the loss function on the validation and training datasets across three epochs depicts that there is fast convergence for both models and both models reach their peak validation performance during Epoch 1. This says that it is sufficient to train for only one epoch using a pre-trained backbone and having a training dataset size of 14,000 instances; otherwise, overfitting may occur during longer training times due to the lack of early stopping.

6. Conclusion: Key Findings and Potential Improvements

6.1 Key Findings

A sentiment classification algorithm based on two fine-tuned pre-trained Transformer models - DistilBERT and RoBERTa - was built on a stratified 20,000 sample subset of the Steam Reviews Clean dataset. Both models performed very well, thereby demonstrating the effectiveness of applying pre-trained models to sentiment classification tasks on relatively small amount of task-specific training data. The major results include:

- RoBERTa outperformed DistilBERT in all selected evaluation metrics: accuracy, precision, recall, F1 (macro), and F1 (weighted).
- The best validation results for both models were obtained in the first epoch, which was likely to have happened because of the powerful initial representation from pre-training stage.
- Despite DistilBERT's inferior performance in comparison with RoBERTa, the latter is still a great option when the speed of prediction and lower computational costs are required.
- Class imbalance brought in a discernible bias towards the majority class of positive sentiment. For both models, macro F1 was less than weighted F1, as expected since class imbalance affected recall for the minority negative class.
- The data-based tokenisation method was quite efficient, with the use of 95th percentile for tokenisation instead of the traditional 512-token limit helping to reduce the number of unnecessary padding and allowing nearly all the reviews. In addition, the tokenisation process took place in batches of 1000 reviews at once instead of performing tokenisation on the whole dataset at once. This method helped prevent any memory overload when handling the vast amount of data.

6.2 Potential Improvements

- **Class weights:** The application of class weights in the CrossEntropyLoss criterion will lead to more significant punishment for the erroneous classification of the negative minority class. Consequently, there will be an improvement in macro-F1, and negative class recall without needing extra data.
- **Early stopping:** Both models exhibited optimum performance at Epoch 1. Further epochs led to slight overfitting. Therefore, early stopping can be applied by using validation loss to avoid wastage of computational resources.
- **Bigger models:** Testing a larger architecture such as RoBERTa-large and DeBERTa (He et al., 2021), despite having high requirements for memory and computing resources, will likely produce even better results due to the consistent state-of-the-art performance of these models on text classification tasks.

References

1. Adem, S. (2016). steam_reviews_clean. [online] Huggingface.co. Available at: https://huggingface.co/datasets/SirSkandrani/steam_reviews_clean [Accessed 22 Apr. 2026].

2. Sanh, V., Debut, L., Chaumond, J. and Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. [online] arXiv.org. Available at: https://arxiv.org/abs/1910.01108 [Accessed 26 Apr. 2026].

3. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L. and Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. ArXiv. [online] Available at: https://www.semanticscholar.org/reader/077f8329a7b6fa3b7c877a57b81eb6c18b5f87de [Accessed 26 Apr. 2026].

4. Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D. and Smith, N.A. (2020). Don't Stop Pretraining: Adapt Language Models to Domains and Tasks. arXiv:2004.10964 [cs]. [online] Available at: https://arxiv.org/abs/2004.10964 [Accessed 4 May 2026].

5. He, P., Liu, X., Gao, J. and Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. [online] Available at: https://iclr.cc/media/iclr-2021/Slides/2562.pdf [Accessed 4 May 2026].

6. Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. [online] ArXiv. Available at: https://arxiv.org/abs/1810.04805.

7. Loshchilov, I. and Hutter, F. (2017). Decoupled Weight Decay Regularization. arxiv.org. [online] Available at: https://arxiv.org/abs/1711.05101.